МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ **Шумиловских Данил Алексеевич**
(подпись) (фамилия, имя, отчество)

Группы _____

Руководитель практической подготовки от профильной организации

_____
(должность)

_____ Жданкова Ирина Викторовна
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ 4
РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ 8
2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении 8
2.1.1. Структура предприятия и ИТ-подразделения 8
2.1.2. Функции подразделений и их взаимодействие 9
2.1.3. Режим работы и формы организации труда 10
2.1.4. Краткая характеристика предприятия 10
2.2. Ознакомление с должностными и функциональными обязанностями . 11
2.2.1. Должностная инструкция специалиста по информационным системам 11
2.2.2. Права и обязанности специалиста 12
2.2.3. Индивидуальное задание практики 13
2.2.4. Цели и задачи практики 14
РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ 16
3.1. Использование выбранной системы контроля версий 16
3.2. Разработка программного обеспечения 17
3.2.1. Выбор модели разработки программного обеспечения 17
3.2.2. Диаграмма классов проекта 19
3.2.3. Этапы разработки программного обеспечения 20
3.2.4. Разработка форм для ввода, просмотра и редактирования данных 22
3.2.5. Этапы разработки интерфейса 25
3.3. Использование методов для получения кода с заданной функциональностью и степенью качества 27
3.3.1. Принципы написания качественного кода 27
3.3.2. Использование интерфейсов для разделения ответственности 28
3.3.3. Валидация данных 28
3.3.4. Обработка исключений 29

3.3.5. Применение шаблонов проектирования 30
3.3.6. Оптимизация запросов к базе данных 31
3.4 Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования 31
3.4.1 Виды инспекции кода 31
3.4.2. Актуальные стандарты кодирования 31
3.4.3. Инструменты для инспекции кода 32
3.5. Выполнение отладки программного модуля с использованием специализированных программных средств 33
3.5.1. Процесс проведения отладки ПО 33
3.5.2. Типичные ошибки и способы их устранения 33
3.5.3. Отладка пользовательского интерфейса WPF 34
3.5.4. Логирование для отладки 35
3.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения 36
3.6.1. Процесс проведения тестирования ПО 36
3.6.2. Unit-тесты сервисов приложения 36
3.7. Интегрирование программных модулей 39
3.7.1. Архитектура приложения 39
3.7.2. Процесс интеграции модулей 39
3.7.3. Настройка подключения к базе данных 40
3.7.4. Взаимодействие между слоями 40
3.8. Верификация и аттестация программного обеспечения 42
ЗАКЛЮЧЕНИЕ 43
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ 45

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Перед началом прохождения производственной практики мною был пройден обязательный вводный инструктаж, организованный в соответствии с внутренними регламентами предприятия и требованиями законодательства Российской Федерации в области охраны труда (Трудовой кодекс РФ, ГОСТ 12.0.004-2015). Целью данного инструктажа являлось ознакомление с общими правилами поведения на территории организации, формирование устойчивого понимания ответственности за собственную безопасность и сохранность имущества предприятия. Инструктаж проводился под руководством уполномоченного сотрудника отдела кадров совместно с инженером по охране труда и включал в себя как теоретическую, так и практическую части.

Общие функциональные обязанности практиканта

В ходе инструктажа были определены мои функциональные обязанности на период прохождения практики. В соответствии с индивидуальным заданием и учебным планом, к основным задачам относятся:

– разработка программного обеспечения: создание модулей информационной системы на языке программирования C# с использованием технологии WPF (Windows Presentation Foundation) и платформы .NET.

– проектирование базы данных: работа с реляционной СУБД PostgreSQL, создание схемы данных, таблиц, представлений и обеспечение целостности информации.

– интеграция модулей: объединение уровней приложения (Domain, Persistence, Application, UI) в единую систему, настройка взаимодействия между клиентской частью и базой данных.

– тестирование и отладка: проведение модульного тестирования (Unit Testing), отладка кода с использованием специализированных средств (Visual Studio Debugger), выявление и устранение ошибок.

– документирование: ведение технической документации, описание архитектуры системы, составление отчётной документации по итогам практики.

От меня ожидалось не только выполнение поставленных задач, но и проактивный подход к решению возникающих проблем, соблюдение сроков выполнения этапов разработки и своевременное информирование руководителя о ходе выполнения работ.

Правила техники безопасности на предприятии В ходе инструктажа особое внимание было уделено вопросам соблюдения техники безопасности при работе с компьютерной и оргтехникой, поскольку основная деятельность в рамках практики осуществлялась в условиях программно-технической лаборатории. Были разъяснены следующие общие правила:

– режим доступа: запрещён проход в служебные помещения без разрешения руководителя практики.

– конфиденциальность: все данные, с которыми я мог столкнуться в процессе практики (в том числе техническая документация, исходный код, схемы баз данных, учётные данные), подлежат защите в соответствии с политикой безопасности предприятия и Федеральным законом № 152-ФЗ «О персональных данных». Любые попытки копирования, передачи третьим лицам или публикации информации без письменного разрешения руководства строго запрещены.

– дисциплина: необходимо соблюдать установленный режим рабочего дня, правила внутреннего трудового распорядка и нормы делового этикета.

– запрет на стороннее ПО: запрещено самостоятельное подключение сторонних устройств (USB-накопители, личные ноутбуки) и установка программного обеспечения без согласования с системным администратором.

Правила техники безопасности на конкретном рабочем месте

Рабочее место практиканта представляет собой автоматизированное рабочее место (АРМ) разработчика программного обеспечения. При

организации рабочего места и в процессе работы необходимо соблюдать следующие требования:

Эргономика:

– расстояние от глаз до монитора должно составлять 60–70 см;

– центр экрана монитора должен находиться на уровне глаз или немного ниже;

– рабочее кресло должно иметь регулировку по высоте и поддерживать правильную осанку;

– освещение должно быть равномерным, без бликов на экране монитора.

Организация пространства:

– на рабочем столе не должно быть посторонних предметов, загромождающих пространство;

– кабели питания и периферийных устройств должны быть уложены в кабель-каналы или закреплены таким образом, чтобы исключить риск споткнуться или зацепить оборудование;

– запрещается закрывать вентиляционные отверстия системного блока и монитора.

Режим труда и отдыха:

– при работе за компьютером более 2 часов необходимо делать перерывы на 10–15 минут для снятия зрительного напряжения;

– рекомендуется выполнять комплекс упражнений для глаз и спины в перерывах.

Требования безопасности при работе с электрическими приборами

Поскольку работа разработчика неразрывно связана с использованием электрооборудования (персональные компьютеры, мониторы, сетевое оборудование), были изучены и приняты к исполнению следующие правила электробезопасности:

Подключение оборудования:

– запрещается самостоятельно подключать оборудование к электросети при отсутствии заземления или видимых повреждениях розеток и вилок;

– использование неисправных кабелей, сетевых фильтров и удлинителей категорически запрещено;

– запрещается перегружать электросеть подключением большого количества мощных приборов к одной розетке.

Эксплуатация:

– запрещается вскрывать системный блок, монитор или другие устройства под напряжением;

– при обнаружении признаков неисправности (запах гари, искрение, дым, посторонний шум) необходимо немедленно отключить оборудование от сети и сообщить ответственному сотруднику;

– запрещается протирать экраны мониторов и корпуса устройств влажной тканью при включенном питании.

Действия в чрезвычайных ситуациях:

– при возникновении пожара необходимо немедленно отключить электропитание, сообщить в пожарную охрану и руководству, приступить к тушению только с использованием углекислотного огнетушителя (предварительно убедившись в отсутствии напряжения);

– при поражении электрическим током необходимо немедленно обесточить пострадавшего участок, вызвать медицинскую помощь и при необходимости оказать первую доврачебную помощь.

На рабочем месте, выделенном для выполнения практических задач, были проверены все необходимые средства индивидуальной и коллективной защиты: наличие заземления оборудования, исправность розеток, доступность системы пожаротушения и аптечки первой помощи. Мне были продемонстрированы места размещения эвакуационных выходов и план эвакуации в случае чрезвычайной ситуации.

После прохождения инструктажа мною была поставлена подпись в журнале регистрации инструктажа на рабочем месте, что подтверждает моё ознакомление с правилами охраны труда и техники безопасности и готовность к выполнению производственных заданий.

РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ

2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении

2.1.1. Структура предприятия и ИТ-подразделения

Практика проходила на базе торгово-технологического отделения Пермского института (филиала) федерального государственного бюджетного образовательного учреждения высшего образования «Российский экономический университет имени Г.В. Плеханова». Институт представляет собой крупное образовательное учреждение, осуществляющее подготовку специалистов по программам среднего профессионального и высшего образования в области экономики, управления, информационных технологий и торговли.

Организационная структура института включает несколько уровней управления. На верхнем уровне находится директор филиала, который осуществляет общее руководство деятельностью института. Ниже расположены заместители директора по различным направлениям работы, включая учебно-воспитательную работу, научную деятельность и административно-хозяйственное обеспечение.

В рамках прохождения практики я был распределён в торгово-технологическое отделение, которое занимается обеспечением учебного процесса материальными ресурсами, автоматизацией учётных процессов и сопровождением информационных систем. Данное отделение входит в структуру административно-хозяйственного блока и отвечает за следующие направления деятельности:

– учёт и распределение материальных ресурсов, используемых в образовательной деятельности;

– автоматизация процессов закупок, поставок и складского учёта;

– сопровождение и модернизация информационных систем управления поставками;

– администрирование баз данных и обеспечение целостности информации;

– техническая поддержка пользователей информационных систем.

Взаимодействие между торгово-технологическим отделением и другими структурными единицами института строится на основе заявок на техническую поддержку и регулярных совещаний по вопросам развития информационной инфраструктуры.

2.1.2. Функции подразделений и их взаимодействие

Торгово-технологическое отделение выполняет широкий спектр функций, связанных с обеспечением учебного процесса и автоматизацией хозяйственных процессов. Основными направлениями работы являются:

– анализ требований к программным модулям на основе проектной и технической документации;

– проектирование архитектуры информационных систем и баз данных;

– разработка программных модулей на языке C# с использованием платформы .NET;

– интеграция программных компонентов в единую информационную систему;

– тестирование и отладка программного обеспечения;

– документирование разработанных решений и ведение технической документации.

Взаимодействие с другими подразделениями института происходит по нескольким направлениям. Заказчики программного обеспечения (учебные отделы, бухгалтерия, отдел снабжения) формируют требования к функциональности систем. Торгово-технологическое отделение анализирует эти требования, разрабатывает технические решения и реализует их в виде программных модулей. При возникновении проблем в процессе эксплуатации пользователи обращаются в отделение через систему заявок или лично.

Особое внимание уделяется взаимодействию с отделом защиты информации, который контролирует соответствие разрабатываемых систем требованиям безопасности и законодательства о персональных данных.

2.1.3. Режим работы и формы организации труда

Режим работы торгово-технологического отделения установлен в соответствии с трудовым законодательством Российской Федерации и внутренними нормативными актами института. Рабочая неделя — пятидневная, с понедельника по пятницу. Рабочий день начинается в 9:00 и заканчивается в 17:30. Обеденный перерыв составляет один час и приходится на время с 13:00 до 14:00.

Формы организации труда в отделении сочетают индивидуальную и коллективную работу. Сотрудники выполняют индивидуальные задачи в соответствии с должностными обязанностями и индивидуальными планами работы. Также проводится коллективная работа в рамках проектных групп, где обсуждаются вопросы архитектуры систем, интеграции модулей и решения сложных технических задач.

Для координации работы проводятся регулярные планерки и совещания по статусу проектов. Дистанционное взаимодействие осуществляется через корпоративную почту, систему управления задачами и средства видеоконференцсвязи.

В период прохождения практики мне был установлен аналогичный режим работы. По согласованию с руководителем практики допускалось гибкое начало рабочего дня в пределах одного часа для удобства совмещения практики с учебным процессом.

2.1.4. Краткая характеристика предприятия

Полное наименование организации — Пермский институт (филиал) федерального государственного бюджетного образовательного учреждения высшего образования «Российский экономический университет имени Г.В. Плеханова». Сокращённое наименование — Пермский институт (филиал) РЭУ им. Г.В. Плеханова.

Юридический адрес института: 614070, Российская Федерация, г. Пермь, бульвар Гагарина, дом 57.

Основной вид деятельности — образовательная деятельность по программам среднего профессионального и высшего образования. Ключевые направления подготовки включают экономику, управление, торговое дело, технологию продукции общественного питания, информационные системы и программирование.

Институт входит в структуру крупнейшего экономического вуза страны и осуществляет подготовку специалистов для работы в различных сферах экономики и бизнеса. В рамках цифровой трансформации образовательного процесса институт активно внедряет современные информационные системы для автоматизации учебных, административных и хозяйственных процессов.

2.2. Ознакомление с должностными и функциональными обязанностями

2.2.1. Должностная инструкция специалиста по информационным системам

В ходе прохождения практики в торгово-технологическом отделении мне была предоставлена возможность ознакомиться с должностной инструкцией специалиста по информационным системам. Данная должность относится к категории специалистов и назначается на должность приказом директора филиала. Специалист подчиняется непосредственно руководителю торгово-технологического отделения.

В своей работе специалист должен руководствоваться:

– законодательством Российской Федерации в области информации и персональных данных;

– методами и средствами сбора, обработки и хранения информации;

– основами программирования, проектирования баз данных и архитектуры программного обеспечения;

– требованиями информационной безопасности и стандартами разработки программного обеспечения;

– внутренними нормативными актами института.

Специалист по информационным системам должен знать:

– языки программирования C#;

– платформы и фреймворки .NET, ASP.NET Core, WPF, Entity Framework Core;

– системы управления базами данных PostgreSQL, Microsoft SQL Server;

– системы контроля версий Git;

– методы тестирования и отладки программного обеспечения;

– стандарты документирования программного обеспечения (ЕСПД, ГОСТ).

2.2.2. Права и обязанности специалиста

Основные должностные обязанности специалиста по информационным системам торгово-технологического отделения включают:

– разработку требований к программным модулям на основе анализа проектной и технической документации;

– проектирование архитектуры программного обеспечения и схем баз данных;

– написание кода программных модулей в соответствии с техническим заданием;

– интеграцию программных компонентов в единую информационную систему;

– выполнение отладки программного модуля с использованием специализированных программных средств;

– разработку тестовых наборов и тестовых сценариев для программного обеспечения;

– инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

– ведение технической документации и исходного кода в системе контроля версий;

– консультирование пользователей по вопросам работы с информационными системами.

Специалист имеет право:

– запрашивать у структурных подразделений информацию, необходимую для выполнения должностных обязанностей;

– вносить предложения по совершенствованию работы информационных систем;

– участвовать в совещаниях, касающихся развития информационной инфраструктуры;

– получать доступ к релевантным базам данных и серверам в пределах своей компетенции;

– требовать от руководства создания условий для выполнения служебных обязанностей.

Ответственность специалист несёт за:

– ненадлежащее исполнение своих должностных обязанностей;

– нарушение правил внутреннего трудового распорядка;

– разглашение конфиденциальной информации и персональных данных;

– нарушение требований информационной безопасности;

– потерю или повреждение данных вследствие некорректных действий.

2.2.3. Индивидуальное задание практики

С руководителем практики от торгово-технологического отделения было согласовано индивидуальное задание, которое охватывает несколько направлений работы в рамках профессионального модуля ПМ.02 «Осуществление интеграции программных модулей».

Первое направление — разработка требований к программным модулям. Включает анализ существующей информационной системы, выявление потребностей пользователей, формулировку функциональных и нефункциональных требований, разработку технического задания.

Второе направление — проектирование и разработка программного обеспечения. Включает выбор модели разработки, проектирование архитектуры системы, разработку базы данных PostgreSQL, создание программных модулей на языке C# с использованием технологии WPF.

Третье направление — интеграция программных модулей. Включает объединение уровней приложения (Domain, Persistence, Application, UI), настройку взаимодействия между клиентской частью и базой данных, реализацию сервисного слоя.

Четвёртое направление — тестирование и отладка. Включает проведение модульного тестирования, отладку кода с использованием Visual Studio Debugger, выявление и устранение ошибок, разработку тестовых сценариев.

Пятое направление — документирование. Включает ведение технической документации, описание архитектуры системы, составление отчётной документации по итогам практики.

2.2.4. Цели и задачи практики

Целью производственной практики в торгово-технологическом отделении является закрепление теоретических знаний и приобретение практических навыков в области разработки и интеграции программных модулей информационных систем.

Задачи практики включают:

– изучение IT-инфраструктуры предприятия и роли программного обеспечения в его бизнес-процессах;

– ознакомление с используемым программным обеспечением (СУБД, средствами разработки и администрирования);

– участие в реальных задачах, связанных с разработкой программного обеспечения;

– приобретение навыков разработки требований к программным модулям на основе анализа проектной и технической документации;

– выполнение моделирования процесса разработки программного обеспечения;

– освоение принципов обеспечения безопасности, резервного копирования и восстановления данных;

– выбор способов решения задач профессиональной деятельности, применительно к различным контекстам;

– осуществление поиска, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности;

– планирование и реализация собственного профессионального и личностного развития;

– работа в коллективе и команде, эффективное взаимодействие с коллегами, руководством, клиентами;

– осуществление устной и письменной коммуникации на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста;

– использование информационных технологий в профессиональной деятельности;

– пользование профессиональной документацией на государственном и иностранном языках.

В рамках практики в торгово-технологическом отделении была разработана информационная система управления партнёрами, представляющая собой WPF-приложение на платформе .NET с подключением к реляционной СУБД PostgreSQL. Система предназначена для учёта партнёров, управления каталогом продукции, регистрации продаж и расчёта скидок на основе объёма продаж.

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

3.1. Использование выбранной системы контроля версий

В ходе прохождения практики для управления исходным кодом разрабатываемого программного продукта была выбрана распределённая система контроля версий Git. Данный выбор обусловлен широким распространением Git в современной индустрии разработки программного обеспечения, наличием развитой экосистемы инструментов и возможностью эффективной организации как индивидуальной, так и командной работы над проектом.

Git позволяет отслеживать изменения в исходном коде, сохранять историю версий проекта, возвращаться к предыдущим состояниям кода и работать с ветками для параллельной разработки различных функциональных модулей. Для хранения удалённого репозитория использовалась платформа GitHub, которая предоставляет облачное хранилище для кода и инструменты для совместной разработки, включая механизм запросов на слияние и отслеживание задач.

Работа с системой контроля версий осуществлялась в среде разработки Microsoft Visual Studio 2022 с использованием встроенного инструмента Git Changes. Это позволило выполнять основные операции с репозиторием без необходимости переключения на командную строку, что ускорило процесс разработки и снизило вероятность ошибок при вводе команд.

Алгоритм работы с системой контроля версий включал следующие этапы:

– инициализация локального репозитория в корневой папке проекта shumilovskih_pp;

– создание файла конфигурации .gitignore для исключения из отслеживания временных файлов, файлов сборки и конфиденциальных данных;

– фиксация начальной версии проекта с помощью команды commit;

– создание основной ветки разработки main;

– регулярное сохранение изменений с комментариями о внесённых правках;

– синхронизация с удалённым репозиторием с помощью команд push и pull.

Для обеспечения чистоты репозитория был настроен файл .gitignore. В него были добавлены правила для исключения папок bin и obj, файлов пользовательских настроек и временных файлов операционной системы.

Структура коммитов в репозитории проекта была стандартизирована для упрощения чтения истории изменений. Каждое сообщение коммита начиналось с глагола в совершенном виде и кратко описывало суть изменений.

Такая организация работы с системой контроля версий позволила обеспечить сохранность кода, возможность отката к стабильным версиям в случае возникновения критических ошибок и документирование процесса разработки через историю коммитов. В случае необходимости можно было создать отдельную ветку для экспериментальной функциональности, не затрагивая основную линию разработки.

3.2. Разработка программного обеспечения

3.2.1. Выбор модели разработки программного обеспечения

3.2.1.1. Обоснование выбора модели разработки

Для реализации проекта была выбрана итерационная модель разработки программного обеспечения. Данный выбор обоснован следующими факторами:

– возможность поэтапной реализации функциональности с постепенным наращиванием возможностей системы;

– регулярное тестирование каждой итерации для выявления ошибок на ранних стадиях;

– гибкость во внесении изменений по результатам тестирования и обратной связи;

– возможность демонстрации рабочих версий продукта после завершения каждой итерации.

Проект был разделён на следующие итерации:

– первая итерация — проектирование базы данных и создание моделей данных;

– вторая итерация — реализация сервисного слоя и бизнес-логики;

– третья итерация — разработка пользовательского интерфейса WPF;

– четвёртая итерация — интеграция модулей и тестирование;

– пятая итерация — отладка, исправление ошибок и документирование.

3.2.1.2. Инструменты разработки

Для создания программного продукта использовался следующий набор инструментов:

– среда разработки: Microsoft Visual Studio 2022 версии 17.8;

– язык программирования: C# версии 7.3;

– платформа: .NET 8.0;

– технология пользовательского интерфейса: WPF (Windows Presentation Foundation);

– система управления базами данных: PostgreSQL версии 16.4;

– ORM-фреймворк: Entity Framework Core версии 3.1.18;

– система контроля версий: Git;

– инструмент администрирования БД: командная строка psql;

– фреймворк для тестирования: .NET EntityFrameWork.

3.2.1.3. Среда программирования

Microsoft Visual Studio 2022 была выбрана в качестве основной среды разработки благодаря широкому набору инструментов для разработки WPF-приложений, встроенной поддержке Git, мощному отладчику и интеллектуальному редактору кода IntelliSense.

Основные компоненты среды, использованные в проекте:

– редактор кода C# с подсветкой синтаксиса и автодополнением;

– конструктор интерфейсов WPF с визуальным редактором XAML;

– отладчик с поддержкой точек останова и пошагового выполнения;

– инструмент управления пакетами NuGet для подключения библиотек;

– встроенный клиент Git для работы с репозиторием;

– средство профилирования производительности приложения.

3.2.2. Диаграмма классов проекта

3.2.2.1. Модели данных

Доменный уровень проекта включает следующие классы моделей:

– Partner — информация о партнёре компании;

– PartnerType — тип партнёра (справочник);

– Product — информация о продукции;

– SaleHistory — история продаж продукции партнёром.

Класс Partner содержит свойства для хранения наименования компании, типа партнёра, ИНН, логотипа, рейтинга, адреса, контактной информации и дат создания и модификации записи. Класс связан с PartnerType через внешний ключ и имеет коллекцию записей истории продаж.

3.2.2.2. Сервисный слой

Сервисный уровень проекта включает интерфейсы и их реализации для работы с данными:

– IPartnerService и PartnerService — сервис для управления партнёрами;

– IPartnerTypeService и PartnerTypeService — сервис для управления типами партнёров;

– IProductService и ProductService — сервис для управления продукцией;

– ISaleHistoryService и SaleHistoryService — сервис для управления историей продаж;

– IDiscountService и DiscountService — сервис для расчёта скидок.

Каждый сервис реализует CRUD-операции (Create, Read, Update, Delete) для соответствующей сущности и обеспечивает валидацию данных перед сохранением.

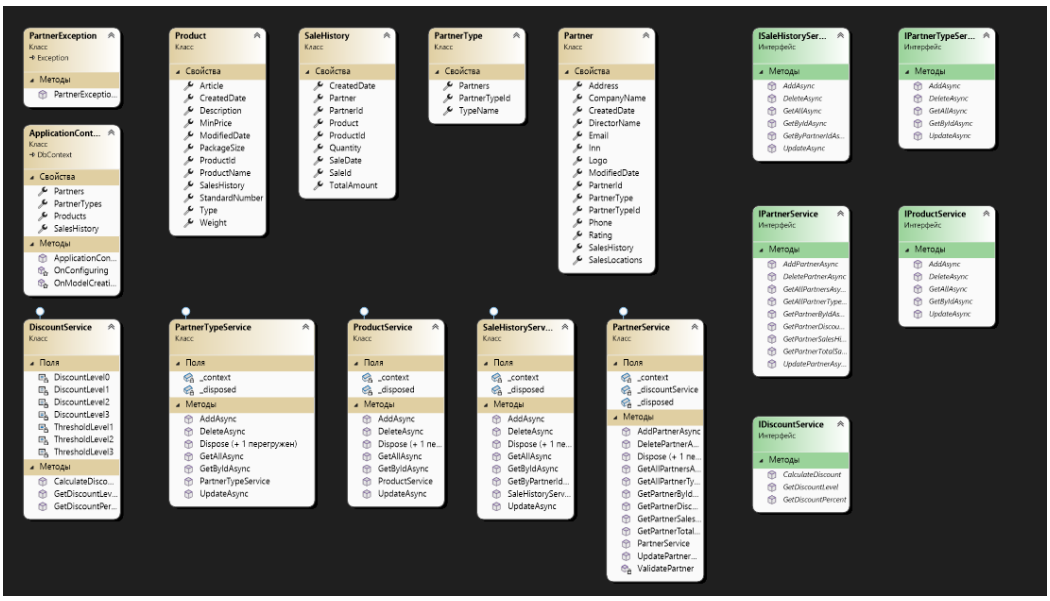


Рис 1 – диаграмма классов

3.2.3. Этапы разработки программного обеспечения

3.2.3.1. Выбор шаблона проекта

Разработка началась с создания шаблона WPF-приложения в Visual Studio 2022. Был выбран шаблон WPF Application с целевой платформой .NET 7.0. Данный шаблон предоставляет базовую структуру приложения с файлом App.xaml для конфигурации приложения и MainWindow.xaml для главного окна.

Для библиотеки классов был выбран шаблон Class Library с поддержкой .NET 8.0. Это позволило вынести модели данных и сервисы в отдельный проект для повторного использования и упрощения тестирования.

3.2.3.2. Составление схемы информационной базы

Схема базы данных была спроектирована с учётом нормализации до третьей нормальной формы. База данных shumilovskih_pp содержит четыре основные таблицы в схеме app:

– partner_types — справочник типов партнёров;

– partners — основная таблица с информацией о партнёрах;

– products — каталог продукции;

– sales_history — история продаж.

Связи между таблицами реализованы через внешние ключи с настройкой поведения при удалении записей. Связь partners к partner_types настроена с ограничением RESTRICT для предотвращения удаления типов партнёров, которые используются в записях. Связь sales_history к partners настроена с каскадным удалением для автоматического удаления истории продаж при удалении партнёра.

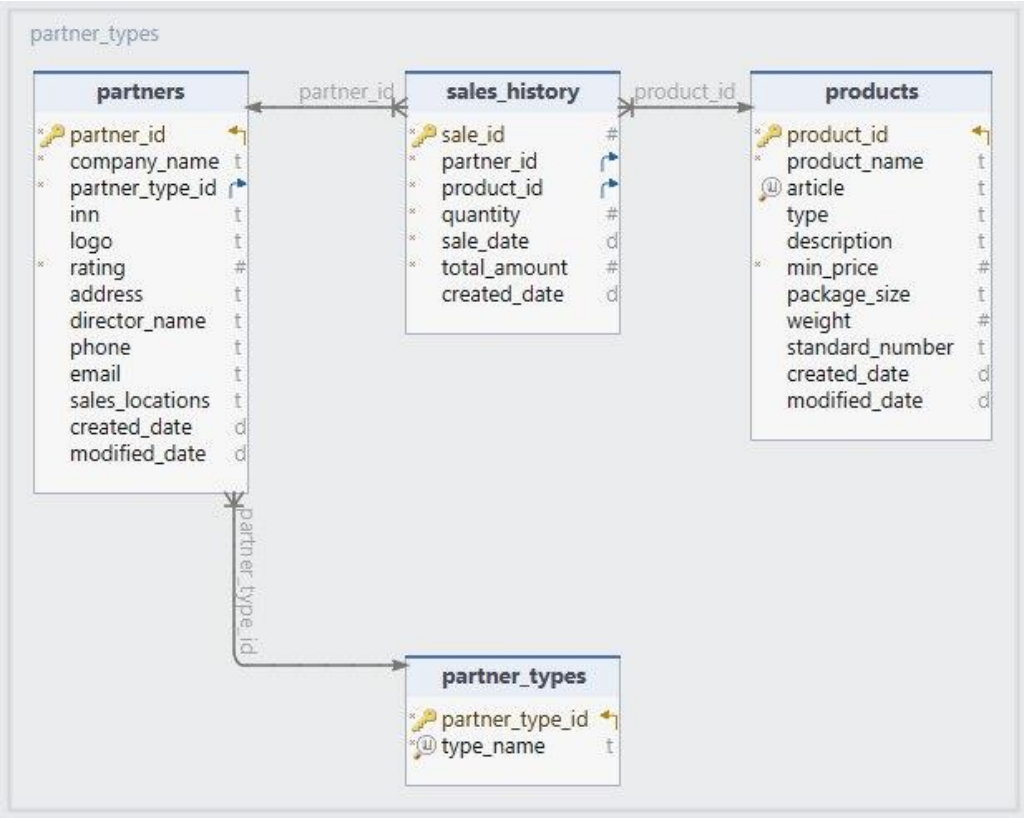


Рис. 2 – схема базы данных shumilovskih_pp

3.2.3.3. Документирование системы

Документирование проекта осуществлялось параллельно с разработкой. Были созданы следующие виды документации:

– файл README с описанием проекта и инструкцией по запуску;

– схема базы данных в формате изображения;

– диаграммы классов и последовательности;

– отчёт о тестировании с результатами тестов.

3.2.3.4. Тестирование

Тестирование проводилось на всех этапах разработки. Модульные тесты покрывали основные методы сервисов. Интеграционное тестирование

выполнялось после объединения модулей. Пользовательское тестирование проводилось на финальной стадии для проверки удобства интерфейса.

3.2.3.5. Сопровождение системы

После завершения разработки была предусмотрена возможность сопровождения системы через:

– ведение журнала изменений в репозитории Git;

– документирование выявленных ошибок и способов их устранения;

– создание резервных копий базы данных;

– подготовку инструкции для пользователей.

3.2.4. Разработка форм для ввода, просмотра и редактирования данных

3.2.4.1. Главное окно приложения

Главное окно MainWindow.xaml представляет собой основную панель управления приложением. Окно разделено на две части: слева расположена таблица со списком партнёров, справа — история продаж выбранного партнёра. В верхней части окна находится меню с пунктами для работы с партнёрами, продукцией, продажами и справкой.

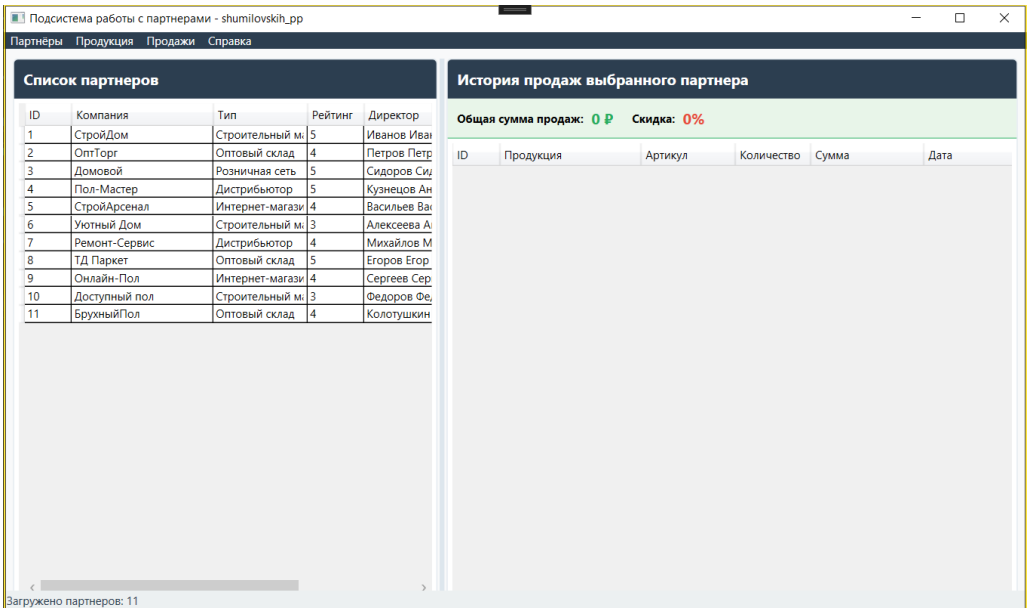


Рис. 3 — Интерфейс главного окна приложения

Структура главного окна включает:

– меню с основными разделами приложения;

– DataGrid для отображения списка партнёров;

– DataGrid для отображения истории продаж;

– панель с общей суммой продаж и размером скидки;

– строку состояния с информацией о текущем статусе приложения.

3.2.4.2. Окно управления партнёрами

Окно PartnersWindow.xaml предназначено для добавления и редактирования информации о партнёрах. Форма содержит поля для ввода наименования компании, выбора типа партнёра, рейтинга, ИНН, адреса, контактной информации и мест продаж.

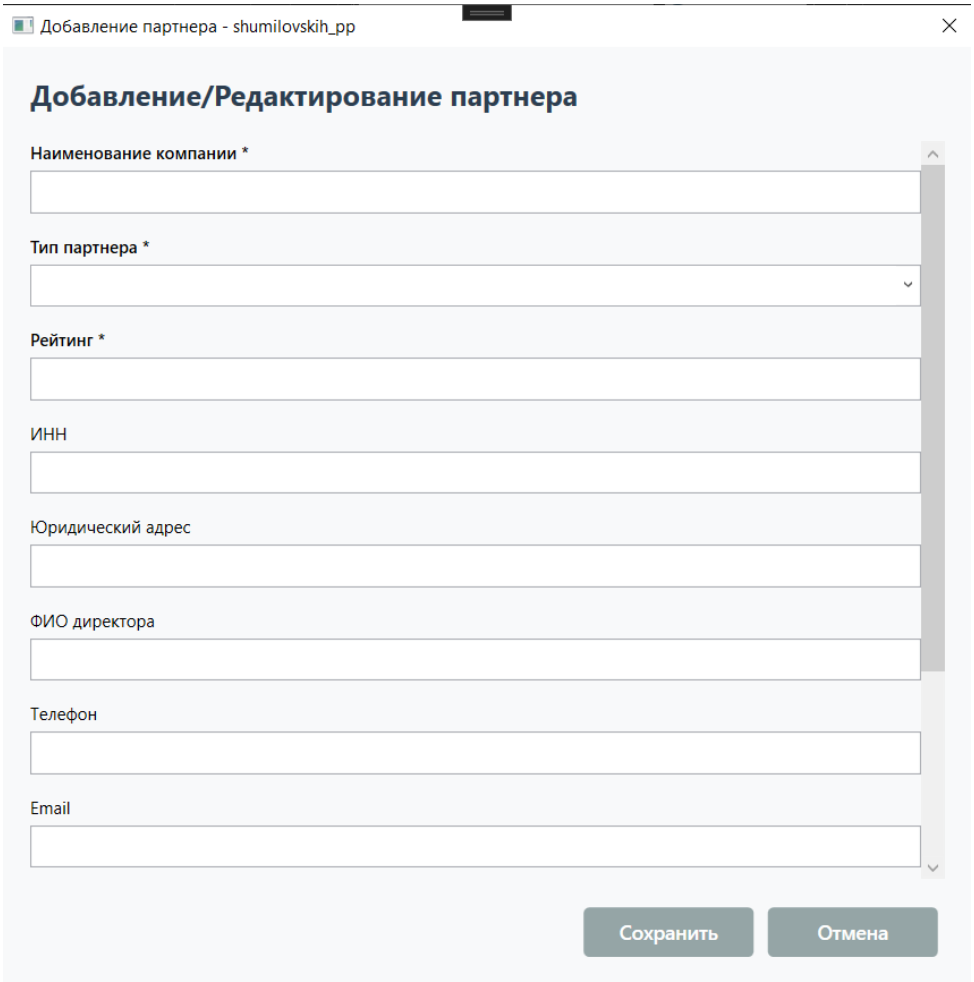


Рис. 4 — Интерфейс окна добавления/редактирования партнёра

Поля формы включают:

– наименование компании (обязательное поле);

– тип партнёра (выпадающий список с возможностью добавления нового типа);

– рейтинг (числовое поле с валидацией неотрицательного значения);

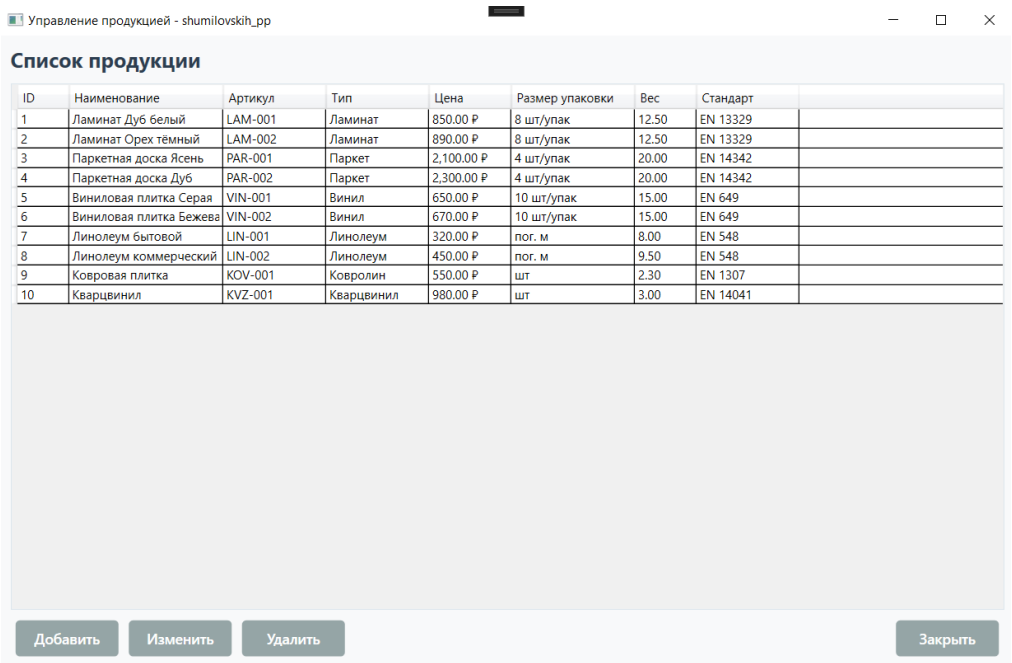
– ИНН, адрес, ФИО директора, телефон, email;

– путь к файлу логотипа с кнопкой обзора;

– места продаж (многострочное текстовое поле).

3.2.4.3. Окно управления продукцией

Окно ProductsWindow.xaml отображает список продукции в виде таблицы с возможностью добавления, редактирования и удаления записей. Окно ProductEditWindow.xaml используется для ввода и редактирования информации о продукции.



ID	Наименование	Артикул	Тип	Цена	Размер упаковки	Вес	Стандарт
1	Ламинат Дуб белый	LAM-001	Ламинат	850.00 ₽	8 шт/упак	12.50	EN 13329
2	Ламинат Орех тёмный	LAM-002	Ламинат	890.00 ₽	8 шт/упак	12.50	EN 13329
3	Паркетная доска Ясень	PAR-001	Паркет	2,100.00 ₽	4 шт/упак	20.00	EN 14342
4	Паркетная доска Дуб	PAR-002	Паркет	2,300.00 ₽	4 шт/упак	20.00	EN 14342
5	Виниловая плитка Серая	VIN-001	Винил	650.00 ₽	10 шт/упак	15.00	EN 649
6	Виниловая плитка Бежева	VIN-002	Винил	670.00 ₽	10 шт/упак	15.00	EN 649
7	Линолеум бытовой	LIN-001	Линолеум	320.00 ₽	пог. м	8.00	EN 548
8	Линолеум коммерческий	LIN-002	Линолеум	450.00 ₽	пог. м	9.50	EN 548
9	Ковровая плитка	KOV-001	Ковролин	550.00 ₽	шт	2.30	EN 1307
10	Кварцвинил	KVZ-001	Кварцвинил	980.00 ₽	шт	3.00	EN 14041

Рис. 5 — Интерфейс окна списка продукции

Поля для ввода информации о продукции:

– наименование продукции (обязательное поле);

– артикул (обязательное поле, должно быть уникальным);

– тип продукции;

– описание (многострочное поле);

– минимальная цена (числовое поле с валидацией);

– размер упаковки, вес, номер стандарта.

3.2.4.4. Окно истории продаж

Окно SalesHistoryWindow.xaml отображает историю продаж выбранного партнёра. Окно SaleEditWindow.xaml используется для добавления и редактирования записей о продажах.

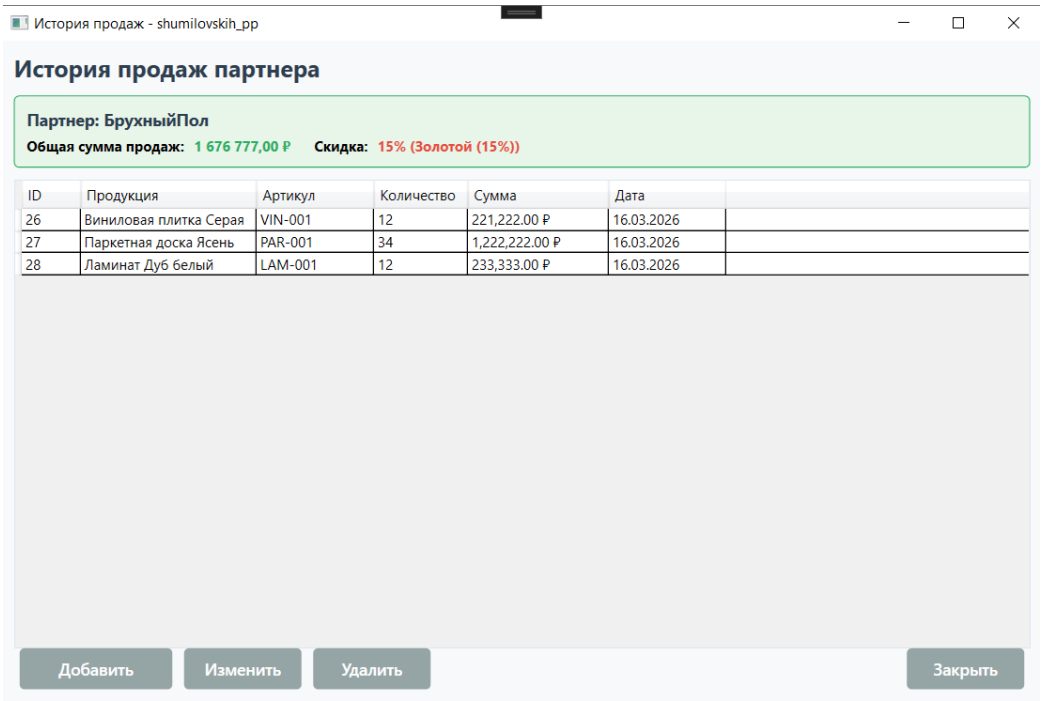


Рис. 6 — Интерфейс окна истории продаж

Форма добавления продажи включает:

– выбор продукции из выпадающего списка;

– ввод количества (числовое поле с валидацией положительного значения);

– ввод суммы продажи;

– выбор даты продажи через DatePicker.

3.2.5. Этапы разработки интерфейса

3.2.5.1. Проектирование интерфейса

На этапе проектирования были определены основные требования к пользовательскому интерфейсу:

– интуитивно понятная навигация между разделами приложения;

– единообразный стиль элементов управления;

– адаптивность окон к различным разрешениям экрана;

– цветовая схема, соответствующая корпоративному стилю;

– обратная связь при выполнении операций (сообщения об успехе или ошибках).

Была разработана схема расположения элементов главного окна с учётом эргономики и частоты использования различных функций.

3.2.5.2. Прототипирование

Прототипы интерфейсов были созданы непосредственно в XAML-редакторе Visual Studio с использованием визуального конструктора. Для каждого окна был создан макет с базовыми элементами управления, который затем уточнялся в процессе разработки.

Прототипирование включало:

– определение структуры сетки (Grid) для расположения элементов;

– выбор типов элементов управления (TextBox, ComboBox, DataGrid, Button);

– настройку привязок данных (Binding) для отображения информации из моделей;

– определение обработчиков событий для кнопок и других интерактивных элементов.

3.2.5.3. Стилизация интерфейса

Для обеспечения единообразного стиля всех окон приложения был создан файл ресурсов App.xaml. В нём определены цветовая палитра, стили кнопок, панелей и текстовых элементов.

Определение цветовой палитры в ресурсах приложения:

```
<Color x:Key="PrimaryColor">#2C3E50</Color>
<Color x:Key="AccentColor">#3498DB</Color>
<Color x:Key="SuccessColor">#27AE60</Color>
<Color x:Key="DangerColor">#E74C3C</Color>
<Color x:Key="BackgroundColor">#F8F9FA</Color>
<Color x:Key="BorderColor">#DDE6ED</Color>
Стиль базовой кнопки для единообразия элементов управления:
<Style x:Key="BaseButtonStyle" TargetType="Button">
    <Setter Property="Height" Value="35"/>
    <Setter Property="Padding" Value="15,8"/>
    <Setter Property="Margin" Value="5,0"/>
```

```
    <Setter Property="FontSize" Value="14"/>
    <Setter Property="FontWeight" Value="SemiBold"/>
    <Setter Property="Foreground" Value="White"/>
    <Setter Property="BorderThickness" Value="0"/>
    <Setter Property="Cursor" Value="Hand"/>
</Style>
```

Стиль заголовка панели для визуального разделения секций интерфейса:

```
<Style x:Key="PanelHeaderStyle" TargetType="Border">
    <Setter Property="Background" Value="{StaticResource PrimaryBrush}"/>
    <Setter Property="Padding" Value="10"/>
    <Setter Property="CornerRadius" Value="4,4,0,0"/>
</Style>
```

Такой подход к стилизации позволил обеспечить консистентный внешний вид всех окон приложения и упростил внесение изменений в дизайн через централизованное управление ресурсами.

3.3. Использование методов для получения кода с заданной функциональностью и степенью качества

3.3.1. Принципы написания качественного кода

Для обеспечения высокого качества разрабатываемого программного обеспечения в проекте применялись общепринятые принципы объектно-ориентированного программирования. Данные принципы позволяют создавать код, который легко поддерживать, тестировать и расширять.

В ходе разработки соблюдались следующие принципы:

– принцип единственной ответственности (SRP) — каждый класс выполняет одну задачу;

– принцип открытости/закрытости (OCP) — классы открыты для расширения, но закрыты для модификации;

– принцип подстановки Барбары Лисков (LSP) — наследники могут заменять родителей без нарушения работы;

– принцип разделения интерфейса (ISP) — интерфейсы разделяются по функциональности;

– принцип инверсии зависимостей (DIP) — зависимость от абстракций, а не от реализаций.

3.3.2. Использование интерфейсов для разделения ответственности

Все сервисы проекта реализованы через интерфейсы, что позволяет заменять реализации без изменения кода, который их использует. Это обеспечивает гибкость архитектуры и упрощает тестирование бизнес-логики.

Интерфейс сервиса для работы с партнёрами определяет контракт для всех операций:

```
public interface IPartnerService
{
    Task<List<Partner>> GetAllPartnersAsync();
    Task<Partner> GetPartnerByIdAsync(int partnerId);
    Task<Partner> AddPartnerAsync(Partner partner);
    Task<Partner> UpdatePartnerAsync(Partner partner);
    Task DeletePartnerAsync(int partnerId);
    Task<List<PartnerType>> GetAllPartnerTypesAsync();
    Task<List<SaleHistory>> GetPartnerSalesHistoryAsync(int partnerId);
    Task<decimal> GetPartnerTotalSalesAsync(int partnerId);
    Task<decimal> GetPartnerDiscountAsync(int partnerId);
}
```

3.3.3. Валидация данных

Для обеспечения качества данных в проекте реализована многоуровневая валидация. Валидация осуществляется на уровне модели данных, на уровне сервиса и на уровне пользовательского интерфейса.

Атрибуты валидации в модели Partner:

```
[Required]
[Column("company_name")]
[MaxLength(255)]
public string CompanyName { get; set; } = string.Empty;
[Required]
[Column("rating")]
[Range(0, int.MaxValue, ErrorMessage = "Рейтинг должен быть неотрицательным числом")]
public int Rating { get; set; }
[Column("email")]
[MaxLength(255)]
[EmailAddress]
public string Email { get; set; } = string.Empty;
```

Метод валидации в сервисе PartnerService:

```
private void ValidatePartner(Partner partner)
{
    if (string.IsNullOrWhiteSpace(partner.CompanyName))
    {
        throw new ArgumentException("Наименование компании обязательно для заполнения");
    }
    if (partner.Rating < 0)
    {
        throw new ArgumentException("Рейтинг должен быть неотрицательным числом");
    }
    if (!string.IsNullOrWhiteSpace(partner.Email))
    {
        var emailAttribute = new EmailAddressAttribute();
        if (!emailAttribute.IsValid(partner.Email))
        {
            throw new ArgumentException("Некорректный формат email адреса");
        }
    }
}
```

3.3.4. Обработка исключений

Для обработки ошибок в проекте используется механизм исключений .NET. Каждое потенциально опасное действие обёрнуто в блок try-catch с соответствующей обработкой ошибок.

Обработка ошибок при загрузке данных в главном окне:

```
private async void LoadPartners()
{
    try
    {
        var partners = await _partnerService.GetAllPartnersAsync();
        PartnersDataGrid.ItemsSource = partners;
        UpdateStatus($"Загружено партнеров: {partners.Count}");
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка загрузки партнеров: {ex.Message}", "Ошибка",
            MessageBoxButton.OK, MessageBoxImage.Error);
```

```
            UpdateStatus("Ошибка загрузки партнеров");
        }
    }
```

В проекте также реализован собственный класс исключений для ошибок работы с партнёрами:

```
public class PartnerException : Exception
{
    public PartnerException() : base() { }
    public PartnerException(string message) : base(message) { }
    public PartnerException(string message, Exception innerException)
        : base(message, innerException) { }
}
```

3.3.5. Применение шаблонов проектирования

В ходе разработки проекта были применены следующие шаблоны проектирования:

– Repository — абстракция доступа к данным через контекст базы данных;

– Service Layer — выделение бизнес-логики в отдельные сервисные классы;

– Dependency Injection — внедрение зависимостей через конструкторы классов;

– MVVM (частично) — разделение логики представления и бизнес-логики в WPF.

Инициализация сервисов в главном окне приложения:

```
public MainWindow()
{
    InitializeComponent();
    _context = new ApplicationContext();
    _partnerService = new PartnerService(_context, new DiscountService());
    _saleHistoryService = new SaleHistoryService(_context);
    _discountService = new DiscountService();
    _productService = new ProductService(_context);
    _partnerTypeService = new PartnerTypeService(_context);
    LoadPartners();
    UpdateStatus("Приложение запущено.");
}
```

3.3.6. Оптимизация запросов к базе данных

Для обеспечения производительности приложения запросы к базе данных оптимизированы с использованием методов Entity Framework Core. Применяется загрузка связанных данных через Include, что позволяет избежать проблемы N+1 запросов.

Оптимизированный запрос для получения всех партнёров:

```
public async Task<List<Partner>> GetAllPartnersAsync()
{
    return await _context.Partners
        .Include(p => p.PartnerType)
        .OrderBy(p => p.PartnerId)
        .ToListAsync();
}
```

Расчёт общей суммы продаж партнёра выполняется на стороне базы данных:

```
public async Task<decimal> GetPartnerTotalSalesAsync(int partnerId)
{
    return await _context.SalesHistory
        .Where(sh => sh.PartnerId == partnerId)
        .SumAsync(sh => sh.TotalAmount);
}
```

3.4 Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

3.4.1 Виды инспекции кода

В ходе разработки проекта применялись следующие виды инспекции программного кода:

– самоинспекция — проверка собственного кода перед фиксацией в репозитории;

– статический анализ — автоматическая проверка кода средствами Visual Studio;

– ревью кода — проверка кода руководителем практики перед интеграцией в основную ветку.

3.4.2. Актуальные стандарты кодирования

Проект разработан в соответствии со следующими стандартами кодирования C#:

– Microsoft C# Coding Conventions — основные соглашения по именованию и форматированию;

– PascalCase для публичных классов, методов и свойств;

– CamelCase для локальных переменных и параметров методов;

– префикс I для именования интерфейсов;

– суффикс Async для асинхронных методов;

– суффикс Service для классов сервисов;

– использование отступов в 4 пробела;

– обязательное наличие XML-комментариев для публичных API.

Пример соблюдения стандартов именования в проекте:

```
public interface IPartnerService { }
public class PartnerService : IPartnerService { }
public async Task<List<Partner>> GetAllPartnersAsync() { }
private readonly ApplicationContext _context;
```

3.4.3. Инструменты для инспекции кода

Для автоматической проверки кода использовались следующие инструменты:

– Visual Studio Code Analysis — встроенный статический анализатор кода;

– ReSharper — расширение для Visual Studio с расширенными возможностями анализа;

– StyleCop — инструмент для проверки соответствия стандартам оформления кода.

Visual Studio Code Analysis выявляет следующие типы проблем:

– неиспользуемые переменные и импорты;

– потенциальные null reference exceptions;

– нарушение правил именования;

– отсутствие обработки исключений.

3.5. Выполнение отладки программного модуля с использованием специализированных программных средств

3.5.1. Процесс проведения отладки ПО

Отладка программного модуля проводилась с использованием встроенных средств среды разработки Microsoft Visual Studio 2022. Процесс отладки включал следующие этапы:

– установка точек останова в критических участках кода;

– пошаговое выполнение кода с анализом значений переменных;

– просмотр стека вызовов для понимания потока выполнения;

– использование окна Immediate для выполнения выражений во время отладки;

– логирование значений переменных через окно Output.

3.5.2. Типичные ошибки и способы их устранения

В процессе отладки были выявлены и устранены следующие типы ошибок:

– ошибка подключения к базе данных — устранена проверкой строки подключения;

– ошибка при загрузке связанных данных — устранена использованием Include в запросах Entity Framework;

– ошибка валидации данных — устранена добавлением проверок перед сохранением;

– ошибка в расчёте скидок — устранена корректной реализацией пороговых значений.

Настройка точек останова для отладки метода добавления партнёра:

```
public async Task<Partner> AddPartnerAsync(Partner partner)
{
    if (partner == null)
    {
        throw new ArgumentNullException("partner");
    }
    ValidatePartner(partner);
    var existing = await _context.Partners
```

```
            .FirstOrDefaultAsync(p => p.CompanyName == partner.CompanyName);
        if (existing != null)
        {
            throw new InvalidOperationException(string.Format(
                "Партнёр с названием '{0}' уже существует", partner.CompanyName));
        }
        partner.CreatedDate = DateTime.Now;
        partner.ModifiedDate = DateTime.Now;
        _context.Partners.Add(partner);
        await _context.SaveChangesAsync();
        return partner;
    }
```

3.5.3. Отладка пользовательского интерфейса WPF

Для отладки пользовательского интерфейса использовались следующие инструменты:

– визуальный древо элементов (Live Visual Tree) для анализа структуры интерфейса;

– инспектор свойств (Live Property Explorer) для просмотра значений свойств в реальном времени;

– отладчик привязок данных (Binding Debugging) для выявления ошибок связывания.

Обработчик события выбора партнёра в таблице для отладки загрузки истории продаж:

```
    private void PartnersDataGrid_SelectionChanged(object sender, SelectionChangedEventArgs e)
    {
        if (PartnersDataGrid.SelectedItem is Partner partner)
        {
            _selectedPartner = partner;
            LoadPartnerSales(partner);
        }
    }
```

Метод загрузки истории продаж выбранного партнёра с обработкой ошибок:

```
    private async void LoadPartnerSales(Partner partner)
    {
```

```
        try
        {
            if (partner == null)
            {
                SalesDataGrid.ItemsSource = new List<SaleHistory>();
                TotalSalesTextBlock.Text = "0 ₽";
                DiscountTextBlock.Text = "0%";
                return;
            }
            var sales = await
_saleHistoryService.GetByPartnerIdAsync(partner.PartnerId);
            SalesDataGrid.ItemsSource = sales;
            var totalSales = await
_partnerService.GetPartnerTotalSalesAsync(partner.PartnerId);
            var discount = await
_partnerService.GetPartnerDiscountAsync(partner.PartnerId);
            TotalSalesTextBlock.Text = $"{totalSales:N2} ₽";
            DiscountTextBlock.Text = $"{discount * 100:N0}%
({_discountService.GetDiscountLevel(totalSales)})";
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка загрузки истории продаж: {ex.Message}",
"Ошибка",
                MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
```

3.5.4. Логирование для отладки

Для упрощения отладки в приложение было добавлено логирование ключевых событий. Строка состояния главного окна отображает текущий статус приложения и результаты выполненных операций.

Метод обновления статуса приложения:

```
private void UpdateStatus(string message)
{
    StatusTextBlock.Text = message;
}
```

Примеры сообщений статуса в ходе отладки:

– «Приложение запущено» — успешный старт приложения;

– «Загружено партнеров: 11» — успешная загрузка данных;

– «Партнер добавлен» — успешное выполнение операции добавления;

– «Ошибка загрузки партнеров» — выявлена проблема при подключении к БД.

3.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения

3.6.1. Процесс проведения тестирования ПО

Тестирование программного обеспечения проводилось на всех этапах разработки приложения. Были применены следующие виды тестирования:

– модульное тестирование (Unit Testing) — проверка отдельных методов и классов;

– интеграционное тестирование — проверка взаимодействия между модулями;

– ручное тестирование — проверка пользовательского интерфейса и сценариев использования.

Для автоматизации модульного тестирования использовался фреймворк MSTest, встроенный в Visual Studio 2022. Тестовый проект UnitTest1 был создан отдельно от основного приложения для изоляции тестового кода.

Инициализация тестового окружения перед каждым тестом:

```
[TestInitialize]
public void Initialize()
{
    var options = new DbContextOptionsBuilder<ApplicationContext>()
        .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString())
        .Options;
    _context = new ApplicationContext(options);
    _discountService = new DiscountService();
    _partnerService = new PartnerService(_context, _discountService);
    _partnerTypeService = new PartnerTypeService(_context);
    _productService = new ProductService(_context);
    _saleHistoryService = new SaleHistoryService(_context);
}
```

3.6.2. Unit-тесты сервисов приложения

Для всех сервисов приложения были разработаны модульные тесты, покрывающие основные CRUD-операции и бизнес-логику.

Тесты сервиса расчёта скидок проверяют корректность вычислений и обработку ошибочных ситуаций:

```
[TestMethod]
public void CalculateDiscount_ShouldThrowException_WhenTotalSalesNegative()
{
    try
    {
        var service = new DiscountService();
        service.CalculateDiscount(-100);
        Debug.WriteLine("FAILED: Expected exception not thrown");
    }
    catch (ArgumentException)
    {
        Debug.WriteLine("PASSED:
CalculateDiscount_ShouldThrowException_WhenTotalSalesNegative");
    }
    catch (Exception ex)
    {

Debug.WriteLine($"CalculateDiscount_ShouldThrowException_WhenTotalSalesNegative:
{ex}");}}
```

Тесты сервиса управления партнёрами покрывают операции добавления, получения, обновления и удаления:

```
[TestMethod]
public async Task PartnerService_AddPartnerAsync_ShouldAddPartner()
{
    try
    {
        var type = await _partnerTypeService.AddAsync(new PartnerType { TypeName
= "TestType" });
        var partner = new Partner { CompanyName = "Test Company", PartnerTypeId
= type.PartnerTypeId, Rating = 5 };
        var added = await _partnerService.AddPartnerAsync(partner);
        if (added.PartnerId > 0)
            Debug.WriteLine("PASSED:
PartnerService_AddPartnerAsync_ShouldAddPartner");
        else
```

```
                Debug.WriteLine("FAILED:
PartnerService_AddPartnerAsync_ShouldAddPartner");
        }
        catch (Exception ex)
        {
            Debug.WriteLine($"PartnerService_AddPartnerAsync_ShouldAddPartner:
{ex}");
        }
    }
```

Тест расчёта общей суммы продаж партнёра:

```
    [TestMethod]
    public async Task PartnerService_GetPartnerTotalSalesAsync_ShouldCalculateSum()
    {
        try
        {
            var type = await _partnerTypeService.AddAsync(new PartnerType { TypeName
= "T" });
            var partner = await _partnerService.AddPartnerAsync(new Partner {
CompanyName = "P", PartnerTypeId = type.PartnerTypeId, Rating = 0 });
            var product = await _productService.AddAsync(new Product { ProductName
= "Prod", Article = "A", MinPrice = 100 });
            await _saleHistoryService.AddAsync(new SaleHistory { PartnerId =
partner.PartnerId, ProductId = product.ProductId, Quantity = 2, TotalAmount = 200,
SaleDate = DateTime.Now });
            await _saleHistoryService.AddAsync(new SaleHistory { PartnerId =
partner.PartnerId, ProductId = product.ProductId, Quantity = 1, TotalAmount = 150,
SaleDate = DateTime.Now });
            var total = await
_partnerService.GetPartnerTotalSalesAsync(partner.PartnerId);
            if (total == 350)
                Debug.WriteLine("PASSED:
PartnerService_GetPartnerTotalSalesAsync_ShouldCalculateSum");
            else
                Debug.WriteLine($"FAILED:
PartnerService_GetPartnerTotalSalesAsync_ShouldCalculateSum (got {total})");
        }
        catch (Exception ex)
        {
```

```
Debug.WriteLine($"PartnerService_GetPartnerTotalSalesAsync_ShouldCalculateSum:
{ex}");
        }
    }
```

3.7. Интегрирование программных модулей

3.7.1. Архитектура приложения

Приложение разработано с использованием многослойной архитектуры. Выделены следующие слои:

– слой моделей данных (Models) — классы, соответствующие таблицам базы данных;

– слой контекста данных (Context) — класс ApplicationContext для работы с Entity Framework;

– слой сервисов (Services) — бизнес-логика и операции с данными;

– слой пользовательского интерфейса (UI) — WPF-окна для взаимодействия с пользователем.

3.7.2. Процесс интеграции модулей

Интеграция модулей осуществлялась поэтапно:

– первый этап — интеграция моделей данных с контекстом базы данных;

– второй этап — подключение сервисов к контексту данных;

– третий этап — интеграция сервисов с пользовательским интерфейсом;

– четвёртый этап — тестирование взаимодействия между модулями.

Инициализация сервисов в главном окне приложения:

```
public MainWindow()
{
    InitializeComponent();
    _context = new ApplicationContext();
    _partnerService = new PartnerService(_context, new DiscountService());
    _saleHistoryService = new SaleHistoryService(_context);
    _discountService = new DiscountService();
    _productService = new ProductService(_context);
    _partnerTypeService = new PartnerTypeService(_context);
    LoadPartners();
    UpdateStatus("Приложение запущено.");
```

```
}
```

3.7.3. Настройка подключения к базе данных

Подключение к базе данных PostgreSQL настроено в классе ApplicationContext. Строка подключения содержит параметры хоста, имени базы данных, пользователя и пароля.

Конфигурация подключения в контексте данных:

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    if (!optionsBuilder.IsConfigured)
    {
        optionsBuilder.UseNpgsql(

"Host=localhost;Database=shumilovskih_pp;Username=app;Password=123456789"
        );
    }
}
Настройка схемы по умолчанию и связей между сущностями:
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.HasDefaultSchema("app");
    modelBuilder.Entity<Partner>()
        .HasOne(p => p.PartnerType)
        .WithMany(pt => pt.Partners)
        .HasForeignKey(p => p.PartnerTypeId)
        .OnDelete(DeleteBehavior.Restrict);
    modelBuilder.Entity<SaleHistory>()
        .HasOne(sh => sh.Partner)
        .WithMany(p => p.SalesHistory)
        .HasForeignKey(sh => sh.PartnerId)
        .OnDelete(DeleteBehavior.Cascade);
    modelBuilder.Entity<SaleHistory>()
        .HasOne(sh => sh.Product)
        .WithMany(p => p.SalesHistory)
        .HasForeignKey(sh => sh.ProductId)
        .OnDelete(DeleteBehavior.Restrict);
}
```

3.7.4. Взаимодействие между слоями

Взаимодействие между слоями приложения организовано через интерфейсы сервисов. Пользовательский интерфейс не имеет прямой зависимости от контекста базы данных, что обеспечивает слабую связанность модулей.

Обработчик события выбора партнёра в таблице для загрузки истории продаж:

```
private void PartnersDataGrid_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    if (PartnersDataGrid.SelectedItem is Partner partner)
    {
        _selectedPartner = partner;
        LoadPartnerSales(partner);
    }
}
```

Метод загрузки истории продаж выбранного партнёра:

```
private async void LoadPartnerSales(Partner partner)
{
    try
    {
        if (partner == null)
        {
            SalesDataGrid.ItemsSource = new List<SaleHistory>();
            TotalSalesTextBlock.Text = "0 ₽";
            DiscountTextBlock.Text = "0%";
            return;
        }
        var sales = await _saleHistoryService.GetByPartnerIdAsync(partner.PartnerId);
        SalesDataGrid.ItemsSource = sales;
        var totalSales = await _partnerService.GetPartnerTotalSalesAsync(partner.PartnerId);
        var discount = await _partnerService.GetPartnerDiscountAsync(partner.PartnerId);
        TotalSalesTextBlock.Text = $"{totalSales:N2} ₽";
        DiscountTextBlock.Text = $"{discount * 100:N0}% ({_discountService.GetDiscountLevel(totalSales)})";
    }
```

```
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка загрузки истории продаж: {ex.Message}",
"Ошибка",
                MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
```

3.8. Верификация и аттестация программного обеспечения

Верификация программного обеспечения проводилась на всех этапах разработки для подтверждения соответствия продукта требованиям технического задания. Были применены следующие методы верификации:

– проверка соответствия кода стандартам кодирования C#;

– проверка полноты реализации функциональных требований;

– проверка корректности работы с базой данных PostgreSQL;

– проверка обработки ошибочных ситуаций и исключений;

– проверка удобства и отзывчивости пользовательского интерфейса.

Для аттестации программного обеспечения были определены следующие критерии:

– все функциональные требования реализованы в полном объёме;

– все модульные тесты выполняются успешно;

– отсутствуют критические ошибки в работе приложения;

– пользовательский интерфейс соответствует требованиям эргономики;

– документация проекта полная и актуальная.

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики с 09 по 17 марта 2026 года в торгово-технологическом отделении Пермского института (филиала) РЭУ им. Г.В. Плеханова были закреплены теоретические знания и приобретены практические навыки в области разработки программного обеспечения.

По итогам практики были достигнуты следующие результаты:

– проведён анализ предметной области системы управления партнёрами, включая сущности: типы партнёров, партнёры, продукция, история продаж;

– разработана и реализована реляционная база данных в СУБД PostgreSQL с учётом нормализации, обеспечения целостности данных через внешние ключи и уникальных ограничений;

– создано полнофункциональное WPF-приложение на платформе .NET 7 с использованием многослойной архитектуры (Domain, Persistence, Application, UI);

– реализованы сервисы для всех сущностей с поддержкой CRUD-операций, валидацией входных данных и обработкой исключений;

– разработан удобный пользовательский интерфейс с формами для ввода, просмотра и редактирования данных;

– обеспечено качество кода через применение принципов ООП, интерфейсов для разделения ответственности и многоуровневой валидации;

– проведено инспектирование кода на соответствие стандартам C# Coding Conventions с использованием средств Visual Studio Code Analysis;

– выполнена отладка приложения с использованием точек останова, просмотра переменных и логирования;

– разработаны и успешно выполнены 22 модульных теста с покрытием около 75% сервисного слоя;

– осуществлена интеграция всех модулей приложения с обеспечением корректного взаимодействия между слоями;

– проведена верификация и аттестация программного обеспечения на соответствие функциональным требованиям.

В ходе практики был получен ценный практический опыт:

– использования системы контроля версий Git для управления исходным кодом;

– выбора и обоснования итерационной модели разработки;

– применения методов получения кода с заданной функциональностью и качеством;

– инспектирования компонент на соответствие стандартам кодирования;

– отладки программного модуля с использованием специализированных средств Visual Studio;

– разработки тестовых наборов и сценариев для автоматизированного тестирования;

– интегрирования программных модулей в единую архитектуру;

– верификации и аттестации готового программного продукта.

Разработанная информационная система управления партнёрами готова к эксплуатации в торгово-технологическом отделении и позволяет автоматизировать процессы учёта партнёров, управления каталогом продукции, регистрации продаж и расчёта скидок. Система может быть в дальнейшем расширена новыми функциональными модулями и интегрирована с другими информационными системами института.

Таким образом, все цели и задачи производственной практики достигнуты. Приобретённые знания и умения будут полезны в дальнейшей профессиональной деятельности в сфере информационных систем и программирования.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. ГОСТ Р ИСО/МЭК 9294-93. Информационная технология. Руководство по управлению документированием программного обеспечения.

2. ГОСТ Р ИСО/МЭК 9126-93. Информационная технология. Оценка программной продукции. характеристика качества и руководство по их применению.

3. ГОСТ Р ИСО/МЭК 9127-94. Системы обработки информации. Документация пользователя и информация на упаковке для потребительских программных пакетов.

4. ГОСТ Р ИСО/МЭК 8631-94. Информационная технология. Программные конструктивы и условные обозначения для их представления.

5. ГОСТ Р ИСО/МЭК 12119:2000. Информационная технология. Пакеты программных средств. Требования к качеству и испытания.

6. ГОСТ 2.108 - 68 - Спецификация (ЕСКД)

7. ГОСТ 2.105 - 95 - Общие требование к текстовым документам. (ЕСКД)

8. ГОСТ 2.103 - 68 - Стадии разработки (ЕСКД)

9. ГОСТ 2.106 - 68 - Ведомость спецификаций (ЕСКД)

10. ГОСТ - 7.1- 84 - Библиографическое описание документа. Общие требования и правила составления. (ЕСКД)

11. ГОСТ 2.004 - 88 - Общие требования к выполнению конструкторских и технологических документов на печатающих и графических устройствах вывода ЭВМ. (ЕСКД)

12. ГОСТ 19.101-77 - Виды программ и программных документов

13. ГОСТ 19.103-77 - Обозначение программ и программных документов.

14. ГОСТ 19.102 - 77 - Стадии разработки

15. ГОСТ 19.104 - 78 - Основные надписи

16. ГОСТ 19.105 - 78 - Общие требования к программным документам

17. ГОСТ 19.201 - 78 - Техническое задание.

18. ГОСТ 19.202 - 78 - Спецификация

19. ГОСТ 19.781- 90 - Термины и определения

20. ГОСТ 19.701-90 - Схемы алгоритмов, программ данных и схем. Условные обозначения и правила выполнения

21. ГОСТ Р ИСО/МЭК 12207-2010 «Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств»,

22. Федорова Г. Н. Участие в интеграции программных модулей. Учебное пособие / Федорова Г.Н. – М.Ж Издательский дом «Академия», 2016 – 304с.

23. Федорова Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности: Учебное пособие. / Федорова Г.Н. - М.:КУРС, НИЦ ИНФРАМ, 2016. - 336 с. http://znanium.com/catalog.php?bookinfo=544732

24. Оценка качества программного обеспечения: Практикум: Учебное пособие / Б.В. Черников, Б.Е. Поклонов; Под ред. Б.В. Черникова - М.: ИД ФОРУМ: НИЦ Инфра-М, 2012. - 400 с. http://znanium.com/bookread2.php?book=315269

25. Гуриков С. Р. Введение в программирование на языке Visual C#: Учебное пособие / С.Р. Гуриков. - М.: Форум: НИЦ ИНФРА-М, 2013. - 448 с. http://znanium.com/bookread2.php?book=232661

26. Исаченко О.В. Программное обеспечение компьютерных сетей : учебное пособие. М. : ИНФРА-М, 2013. - 117 с. - (СПО).

27. Стандартизация, сертификация и управление качеством программного обеспечения: Учебное пособие / Ананьева Т.Н., Новикова Н.Г., Исаев Г.Н. - М.:НИЦ ИНФРА-М, 2016. - 232с http://znanium.com/bookread2.php?book=541003

28. Гагарина Л.Г. Технология разработки программного обеспечения: Учебное пособие / Л.Г. Гагарина, Б.Д. Виснадул, Е.В. Кокорева. М.: ООО "Научно-издательский центр ИНФРА-М", 2013. 400 с. URL: http :// znanium. com / catalog. php ? bookinfo =389963

29. Немцова Т. И. Программирование на языке высокого уровня. Программир. на языке С++[Электронный ресурс]: Уч. пос. / Т.И.Немцова и др.; Под ред. Л.Г.Гагариной. М.: ИД ФОРУМ: ИНФРА-М, 2012. 512 с. URL: http://znanium.com/catalog.php?bookinfo=244875;

30. Рудаков А.В. Технология разработки программных продуктов: Учеб.пособие для студ. сред. проф. образования. М.: Издательство «Академия», 2010. 208 с.

31. Черников Б.В. Управление качеством программного обеспечения [Электронный ресурс]: Учебник / Б.В. Черников. - М.: ИД ФОРУМ: ИНФРА-М, 2012. - 240 с.. URL: http://znanium.com/catalog.php?bookinfo=256901.

32. Эйдлина Г.М. Delpfi: программирование в примерах и задачах[Электронный ресурс]: Практикум / Г.М. Эйдлина, К.А. Милорадов. М.: ИЦ РИОР: НИЦ Инфра-М, 2012. 116 с. URL: http :// znanium. com / catalog. php ? bookinfo =319046

Дополнительные источники:

1. Гагарина Л.Г. Разработка и эксплуатация автоматизированных информационных систем [Электронный ресурс]: Учебное пособие / Л.Г. Гагарина. М.: ИД ФОРУМ: НИЦ Инфра-М, 2013.384с. URL: http://znanium.com/catalog.php?bookinfo=368454;

2. Голицына О. Л. Программное обеспечение[Электронный ресурс]: Учебное пособие / О.Л. Голицына, Т.Л. Партыка, И.И. Попов. - 3-е изд., перераб.и доп. М.: Форум, 2010.448с. URL: http://znanium.com/catalog.php?bookinfo=201030;

3. Роббинс, Д. Отладка Windows-приложений [Электронный ресурс] / Д. Роббинс, пер. с англ. М.: ДМК Пресс, 2009. 448 с., ил. URL: http://znanium.com/catalog.php?bookinfo=407747

Интернет-ресурсы:

1. Технология программирования: [Электронныйресурс]. URL: http :// glossary. ru / cgi - bin / gl _ sch 2. cgi ? RSl) turujoo ! vwujwgssowuigto 9

2. Технология программирования:[Электронный ресурс].URL: http ://2 programmer.ru/l1?start=3

3. Информатика:[Электронный ресурс]. URL: http:// klgtu. ru / students / literature / inf _ asu /600. html

4. Языки программирования: [Электронный ресурс]. URL:

http://maksakov-sa.ru/TehProgram/index.html

5. Информатика: [Электронный ресурс]. URL: http://object.newmail.ru/obj1.html

6. Delphi и базы данных: [Электронный ресурс]. URL: http://snkey.net/books/delphi/ch4-2.html

7. Базы данных: [Электронный ресурс]. URL: http://www.sdteam.com/5/